



Chapter 14

Project 4.2: Creating Reports

One easy way to share good-looking results is to use a Jupyter notebook's **Markdown** cells to create a presentation. This chapter will create a “slide deck” that can be shared and presented. We can expand on this to create a PDF report using additional packages like **Jupyter book** or **Quarto**.

In this chapter, we'll look at two important working results of data analysis:

- Slide decks build directly from a Jupyter Lab notebook.
- PDF reports built from notebook data and analysis.

This chapter's project will upgrade an analysis notebook created in the previous chapter to create presentations that can be shared with colleagues. We'll start by looking at the kinds of reports an analyst may need to produce.

14.1 Description

The first dozen chapters in the book created a pipeline to acquire and clean raw data. Once the data is available, we can now do more analytical work on the clean data.

The goal is to transform raw data into information, and possibly even insight to help stakeholders make properly informed decisions.

Analytical results need to be shared to be valuable. A Jupyter

Notebook is a solid basis to create readable, helpful presentations and reports.

We'll start by transforming an analysis notebook into a slide deck. You can then use this slide deck to talk through our key points with stakeholders, providing helpful visuals to back up the information they need to understand. These are common in an enterprise environment. (Some would argue they are too common and contain too much of the wrong kind of details.)

We'll start by looking at creating slide decks and presentations in Jupyter Lab.

14.1.1 Slide decks and presentations

A Jupyter Notebook can be exported to a presentation file. The underlying presentation will be an HTML-based repository of individual pages. **Reveal.js** project is used to control the navigation between pages. See <https://revealjs.com> for more details on how this engine works.

Within the notebook, each cell has properties. The “right sidebar” is the property inspector window, letting us manage these properties. One of the properties is the slide type. This allows the analyst to mark cells for inclusion in a presentation. We'll look at the technical details in *[Preparing slides](#)*.

There are a huge number of guides and tutorials on creating useful, informative presentations. The author likes to focus on three key points:

1. Tell them what you'll tell them. Present a list of topics (or an agenda or an outline). This will often use **Markdown** lists.
2. Tell them. This should proceed from general observations to the specific details of the presentation. This may be a mixture of **Markdown** text and figures.

3. Tell them what you told them. Present a summary of what your message was and the actions you'd like them to take in response. This, too, will often use **Markdown** lists.

What's important here is using the slide presentation to contain the keywords and phrases to help the audience remember the essential points and the call to action. This often means making use of **Markdown** text to emphasize words with a bold or italic font. It can also mean using **Markdown** lists of various kinds.

Another important part is avoiding visual clutter created when trying to cram too many points into a single page. When there are a lot of details, a presentation may not be the best approach to managing all the information. A report document may be more useful than a presentation. A document can provide supplemental details to support a brief presentation as well.

14.1.2 Reports

There's a blurry edge between a presentation and a report. Presentations tend to be shorter and focus on keywords and phrases. Reports tend to be longer and written in complete sentences. A well-organized presentation with complete sentences can be viewed as a brief report. A report with short paragraphs and a lot of figures can look like a presentation.

Markdown formatting provides a lot of capabilities to create publication-quality typeset documents. The technology stack from **Markdown** to HTML to a browser or a PDF rendering engine involves a number of transformation steps to get from simple Unicode text in a notebook cell to a richly detailed rendering. This stack is a first-class part of Jupyter Lab and can be exploited by tools like **Quarto** or **Jupyter{Book}** to create reports.

Not all of the typesetting conventions used by publications are available through **Markdown** source files. For example, some publication style guides will include an abstract section that has narrower margins, and sometimes a smaller font. This can be challenging to implement in the **Markdown** language. Some authors will use a less complex layout that lacks all the visual cues of margin and font size.

The power of HTML and CSS is such that a great many typesetting capabilities are available to the author willing to master the technology stack. The reader is encouraged to explore the capabilities of **Markdown**, HTML, and CSS. The reader is also advised to set realistic goals; a great deal of time can be invested in combining **Markdown** and CSS to achieve typesetting effects that don't enhance a report's message.

It often works out well to put each paragraph into a separate cell. This is not a strict rule: sometimes a group of paragraphs should be put into a single cell.

Top-level headings should often be in cells by themselves. This can make it easier to reorganize content within those headings. Some lower-level headings should be in the cell with their introductory paragraph since the heading and the cell's content are unlikely to be separated.

We might have a cell with a level one heading that looks like this:

```
# Title of our document
```

This cell has only the title and no following text. There will likely be a subheading in the next cell with the introduction to the document.

A lower-level cell might look like this:

```
## Conclusion
```

The various **Anscombe Quartet** series all have consistent means and standard deviations.

This cell has both a level two title, and the introductory text for this section of the document. It uses **Markdown** syntax to show that a particular phrase should have strong emphasis, usually done with a bold font.

In the next sections, we'll talk about the technical approach to adding tools to the Jupyter environment so that the analyst can create presentations or reports.

14.2 Overall approach

We'll talk about the general technical steps to creating presentations and reports in a Jupyter Notebook. For presentations, no additional tools are needed. For some simple reports, the **File** menu offers the ability to save and export a notebook as pure Markdown, as a PDF file, or as a LaTeX document. For more complicated reports, it can help to use supplemental tools that create a more polished final document.

14.2.1 Preparing slides

An HTML-based presentation via **Reveal.js** is a first-class feature of a Jupyter Notebook. The **File** menu offers the ability to save and export a notebook as Reveal.js slides. This will create an HTML file that will display as a presentation.

Within Jupyter, the property inspector is used to set the type of slide for a cell. There's an icon of two meshed gears on the top right side of the page to show the property inspector in the right sidebar. Under the **View** menu, the option to show the right sidebar will also show the property inspector.

There are several choices of **Slide Type** for each cell. The most important two choices are "slide" and "skip".

The “slide” will be displayed as part of the presentation. The “skip” cells will be dropped from the presentation; this is great for computations and data preparation. The other options allow combining cells into a single slide and having subsidiary presentation slides.

Creating **Markdown** content and setting the slide type to “slide” creates the narrative text portion of a presentation. These slides would include title pages, agenda, and key points: all of the prompts and take-away bullet points will be in these kinds of cells.

For data visualizations, we can use **Seaborn** or **PyPlot** to create the figure. The cell output has the slide type set to “slide” in the property inspector to include the visualization.

We can mark the computations, function definitions, and doctest cells with a slide type of **skip**. This will omit these details from the presentation.

The analyst can share the notebook with audience members who want to see the supporting details.

The **Reveal.js** has a huge repertoire of capabilities. Many of these features are available through HTML markup. For example, the auto-animate feature will smoothly transition between cells. Since HTML markup is part of **Markdown**, some familiarity with HTML is required for the use of the most advanced features.

The final step is to use the CLI to convert the notebook to a slide deck. The **File** menu has a **Save and Export Notebook As...** option, but this tends to make all of the code visible. Having visible code can distract from the essential message of the visualizations.

The following command hides the cell input value — the code — from the presentation:

```
jupyter nbconvert --to slides --no-input <notebook.ipynb>
```

Use the name of your notebook in place of `<notebook.ipynb>`. This will create an HTML file with the **Reveal.js** code included.

The overall process has three steps:

1. Edit the notebook.
2. Prepare the presentation. (The terminal tool in Jupyter Lab is ideal for this.)
3. View the presentation to find problems.

This is distinct from the way products like **Keynote** and **PowerPoint** work. When working with Jupyter Lab, there will be a bit of flipping back- and forth between browser windows and the notebook window. Placing windows on each side of the display can be helpful.

Be sure to refresh the browser window after each change to the notebook.

14.2.2 Preparing a report

Creating reports is a first-class part of Jupyter Lab. The **File** menu offers the ability to save and export a notebook as pure Markdown, as a PDF file, or as a LaTeX document.

A tool like **pandoc** can convert a **Markdown** file into a wide variety of desired formats. For output creating using LaTeX formatting, a TeX rendering package is required to create a PDF file from the source. The **TeXLive** project maintains a number of tools useful for rendering LaTeX. For macOS users, the **MacTeX** project offers the required binaries. An online tool like **Overleaf** is also useful for handling LaTeX.

In many cases, more sophisticated processing is required than simply saving a notebook as a pure **Markdown** file. We can add the

Jupyter{Book} tools to our environment. For more information see <https://jupyterbook.org>.

The `jupyter-book` component needs to be added to the `requirements-dev.txt` file so other developers know to install it.

When using **conda** to manage virtual environments, the command might look like the following:

```
% conda install --channel conda-forge jupyter-book
```

When using other tools to manage virtual environments, the command might look like the following:

```
% python -m pip install jupyter-book
```

A Jupyter book can be considerably more complicated than a single notebook file. There will be configuration and **table-of-contents (TOC)** files to structure the overall report. The content can be provided in a mixture of **Markdown**, reStructuredText, and Notebook files. Additionally, an extended version of the **Markdown** language, **MyST**, is available to add a wide variety of semantic markup capabilities.

One way to get started is to use the `jupyter-book create` command to create a template project. This template includes the required `_config.yml` and `_toc.yml` files. It also includes examples of various other files that might be part of a project.

The `_config.yml` file has the title and author information. This is the place to start customizing the content to provide the correct report and author names. Other parts of the configuration may need to be changed, depending on how the report will be published. The built-in assumption is an HTML upload to a public repository. For many reports, this is ideal.

For some enterprise projects, however, reporting to a public repository with links to public GitHub isn't acceptable. For these cases, the `_config.yml` file will have to be changed to correct the repository options to refer to an in-house repository.

It's often helpful to immediately edit the `_toc.yml` file, and start creating the report's outline. Generally, the data and notebooks already exist. The audience is often known, and the key points the audience members need to absorb are clear, permitting the analyst to create the outline, and placeholder documents, right away.

In some cases, the analyst can fill in the outline with notes extracted from the analysis notebooks. This refactoring of content can help to trim working notebooks down to the essential computation and visualization. The narrative text can be segregated into MyST or Markdown files outside the notebooks.

Once the content is in draft form, a book is created with the `jupyter-book build` command. This will use the configuration and TOC file to build the complete document from various sources. The default document is an HTML page.

As of this book's publication date, version 0.15.1 includes a warning that direct PDF production is under development and may have bugs. The more reliable way to create PDFs is to use `Jupyter{Book}` to create a LaTeX file. The OS-native `LaTeX` command can be used to build PDFs. An alternative is to use the `sphinx-jupyterbook-latex` package to wrap the TeXtools that transform the LaTeX to PDF.

This involves a number of moving parts, and the installation can be daunting. Here are some of the steps involved:

1. The source **Markdown** text is converted to LaTeX by Jupyter Book.
2. Some intermediate work may be performed by the `sphinx-jupyterbook-latex` package.

3. The final PDF is created by an OS `latex` command; this is either the **MacTeX** or **TeXLive** installation of the TeX tools.

The CLI build command is the `jupyter-book build` command with an additional option `--builder pdflatex` to specify that Sphinx and the TeX tools are used to render the PDF.

14.2.3 Creating technical diagrams

Technical diagrams, including the wide variety of diagrams defined by the UML, are often challenging to create. Popular presentation tools like **Keynote** and **PowerPoint** have clever drawing tools with lots of built-in shapes and options for positioning those shapes on a slide.

There are several choices for creating diagrams for a presentation:

- Use a separate graphics application to create `.PNG` or `.jpg` files and incorporate the graphics into the document. Many of the diagrams in this book were created with PlantUML, for example. See <https://plantuml.com>.
- Use `matplotlib`, and write code to create the image. This can involve a lot of programming to draw some boxes connected by arrows.

The **PlantWEB** project provides a Python interface to the PlantUML web service. This allows an analyst to work as follows:

1. Create a file with text in the **domain-specific language (DSL)** that describes the image.
2. Render the image with the PlantUML engine to create an `.jpg` file.
3. Import the image into the notebook as an IPython `SVG` object.

The image rendering uses the PlantUML server; this requires an active internet connection. In cases where the analyst might be working offline, the **PlantWEB** documentation suggests using **Docker** to run a lo-

cal service in a local Docker container. This will do diagram rendering quickly without the requirement to connect to the internet.

Having looked at the various technical considerations to create slides and a report, we can emphasize the deliverables for this project.

14.3 Deliverables

There are two deliverables for this project:

- A presentation
- A report

The presentation should be an HTML document using the **Reveal.js** slide deck.

The report should be a PDF document from a single notebook. It should contain the visualization figures and some narrative text explaining the images.

For information on unit testing and acceptance testing of the notebooks, see [***Chapter 13, Project 4.1: Visual Analysis Techniques***](#). This project should build on the previous project. It doesn't involve dramatic new programming. Instead, it involves the integration of a large number of components to create meaningful, useful presentations and reports.

14.4 Summary

In this chapter, we have built two important working results of data analysis:

- Slide decks that can be used as presentations to interested users and stakeholders
- Reports in PDF format that can be distributed to stakeholders

The line between these two is always hazy. Some presentations have a lot of details and are essentially reports presented in small pages.

Some reports are filled with figures and bullet points; they often seem to be presentations written in portrait mode.

Generally, presentations don't have the depth of detail reports do. Often, reports are designed for long-term retention and provide background, as well as a bibliography to help readers fill in missing knowledge. Both are first-class parts of a Jupyter notebook and creating these should be part of every analyst's skills.

This chapter has emphasized the additional tools required to create outstanding results. In the next chapter, we'll shift gears and look at some of the statistical basics of data modeling.

14.5 Extras

Here are some ideas for the reader to add to these projects.

14.5.1 Written reports with UML diagrams

In *[Creating technical diagrams](#)* the process of creating UML diagrams was summarized. The reader is encouraged to use PlantUML to create C4 diagrams for their data acquisition and cleaning pipeline. These `.jpg` files can then be incorporated into a report as Markdown figures.

For more information on the C4 model, see <https://c4model.com>.